

PROGRAMACION ORIENTADA A OBJETOS

Clase2 : Introducción a Java



¿Qué es Java?



Creado en 1995

Desarrollado por **Sun Microsystems**, hoy mantenido por Oracle.
Uno de los lenguajes más usados del mundo.



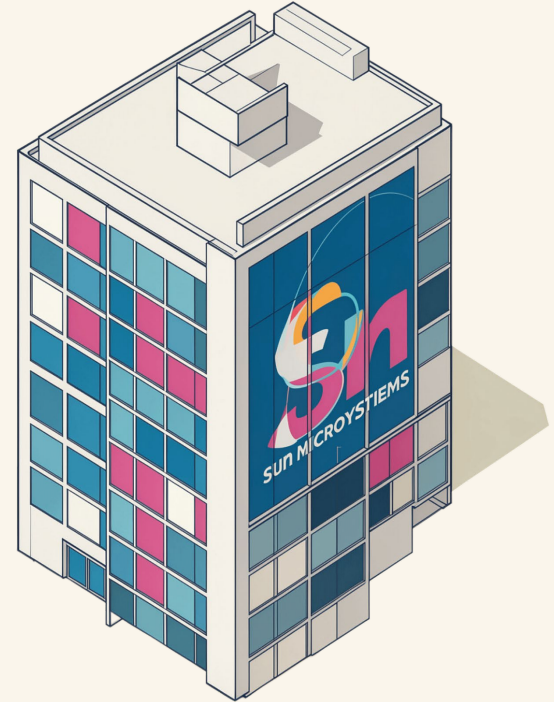
Orientado a objetos, multiplataforma y seguro

Todo se organiza en clases y objetos. Diseñado desde el inicio para ser portable y robusto.



"Write Once, Run Anywhere"

El código compilado corre en **cualquier máquina** que tenga una JVM instalada, sin recompilar.



Características Principales



Orientado a Objetos

Todo se organiza en **clases y objetos** favoreciendo la reutilización y el orden del código.



Portabilidad

Se compila a **bytecode** ejecutable en cualquier JVM, independientemente del sistema operativo.



Tipado Fuerte

Cada variable debe tener un **tipo definido**. Reduce errores en tiempo de ejecución.



Multihilo

Permite ejecutar **múltiples procesos** simultáneamente dentro de una misma aplicación.



Garbage Collector

Gestión **automática de memoria**: libera objetos que ya no se utilizan sin intervención manual.

Herramientas del Ecosistema Java



JDK — Java Development Kit

Incluye el compilador (`javac`), librerías estándar y todas las herramientas necesarias para **desarrollar** en Java.

JRE — Java Runtime Environment

Permite **ejecutar** programas Java sin necesidad de instalar el entorno de desarrollo completo.

JVM — Java Virtual Machine

Interpreta el bytecode y lo ejecuta sobre el sistema operativo anfitrión. Es la clave de la portabilidad.

Tu Primer Programa: ¡Hola, Mundo!

Estructura mínima de un programa Java

Todo programa Java necesita al menos:

- Una **clase pública** con el mismo nombre que el archivo `.java`
- Un método estático llamado `main`

```
public class HolaMundo {  
    public static void main(String[] args) {  
        // Imprimimos un mensaje simple por consola  
        System.out.println("=====");  
        System.out.println("¡Bienvenidos a la Clase 1 de Java!");  
        System.out.println("Entorno IntelliJ configurado correctamente.");  
        System.out.println("=====");  
    }  
}
```

Anatomía del Programa

01

```
public class HolaMundo
```

Define la **clase pública**. En Java, todo el código reside dentro de una clase. El nombre debe coincidir con el nombre del archivo.

02

```
public static void  
main(String[] args)
```

Es el **método principal**: el punto exacto donde la JVM comienza a ejecutar el programa. Siempre debe tener esta firma.

03

```
System.out.println(...)
```

Imprime texto en la **consola estándar** seguido de un salto de línea. Es la instrucción de salida más común en Java.

```
public class HolaMundo {  
    public static void main(String[] args) {  
        // Imprimimos un mensaje simple por consola  
        System.out.println("=====");  
        System.out.println("¡Bienvenidos a la Clase 1 de Java!");  
        System.out.println("Entorno IntelliJ configurado correctamente.");  
        System.out.println("=====");  
    }  
}
```

Tipos de Datos Primitivos

Los tipos primitivos son los bloques de construcción básicos. Java ofrece **8 tipos primitivos** para representar números, caracteres y valores lógicos.

- **Enteros:** byte (1B) · short (2B) · int (4B) · long (8B)
- **Decimales:** float (4B, ~7 decimales) · double (8B, ~15 decimales)
- **Otros:** char (2B, carácter Unicode) · boolean (true/false)

Tipo	Tamaño	Ejemplo
Byte	1 byte	byte b = 10;
Short	2 bytes	short s = 1000;
Int	4 bytes	int n = 100;
Long	8 bytes	long l = 100000L;
Float	4 bytes	float f = 3.14f;
Double	8 bytes	double d = 3.1415;
Char	2 bytes	char c = 'A';
boolean	1 byte	boolean b = true;

Tipos de Referencia

¿Qué almacenan?

No guardan el valor directamente. Almacenan la **dirección de memoria** donde vive el objeto en el Heap.

String

Cadena de texto inmutable.

```
String nombre = "Ana";
```

Arrays

Colección de elementos del mismo tipo.

```
int[] nums = {1, 2, 3};
```

Sintaxis Básica: Variables y Comentarios

Declaración de Variables

```
int edad = 25;  
String nombre = "Luis";  
double precio = 19.99;  
boolean activo = true;
```

Siempre se indica **etipo** seguido del nombre y el valor inicial.

Tipos de Comentarios

```
// Comentario de una línea  
  
/* Comentario  
   multilínea */  
  
/** Comentario JavaDoc  
 *   para documentación  
 */
```

Los comentarios **no se ejecutan**—sirven para documentar y explicar el código.

Palabras Reservadas en Java

Son palabras que el lenguaje usa internamente. **No pueden usarse como nombres de variables, clases o métodos.**



Estructura

`class · interface · abstract · extends · implements`



Acceso

`public · private · protected · static · final`



Control de Flujo

`if · else · for · while · switch · return`



Tipos

`int · double · void · boolean · new`

Clases y Métodos

La clase: plantilla de objetos

Define los **atributos** (datos) y los **métodos** (funciones) que tendrá cada objeto creado a partir de ella.

```
public class Persona {  
    private String nombre;  
  
    public void saludar() {  
        System.out.println(  
            "Hola, soy " + nombre);  
    }  
}
```

El método `saludar()` realiza una acción. Los atributos almacenan el estado del objeto.

Constructores e Inicialización de Objetos

Un **constructor** es un método especial que se ejecuta automáticamente al crear un objeto con `new`. Tiene el mismo nombre que la clase y **no declara tipo de retorno**.

Ejemplo de Constructor

```
public class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```

Puntos Clave

- Mismo nombre que la clase
`Persona(...)` para la clase `Persona`
- Sin tipo de retorno
No se escribe ni `void` ni ningún otro tipo
- Usa `this`
Para distinguir el atributo del parámetro con el mismo nombre

Creando y Usando Objetos

Se usa la palabra clave `new` para **instanciar** una clase. Cada objeto tiene sus propios valores de atributos, independientes de otros objetos de la misma clase.

Crear el objeto

```
Persona p1 = new  
Persona("Ana", 25);
```

Se llama al constructor con los argumentos requeridos.

Llamar un método

```
p1.saludar();
```

Se usa el operador punto `.` para acceder a los métodos del objeto.

Resultado en consola

```
Hola, soy Ana
```

El objeto `p1` ejecuta el método con sus propios datos.

Atributos de Instancia vs. de Clase

Atributo de Instancia

Pertenece **acada objeto individualmente**. Cada instancia tiene su propia copia.

```
private String nombre;  
// Cada Persona tiene su nombre
```

Atributo de Clase (static)

Compartido por todos los objetos de esa clase. Existe aunque no se haya creado ningún objeto.

```
static int contador;  
// Todos comparten el mismo contador
```

Llamada a método estático

Los métodos **static** se invocan con el nombre de la clase, no con una instancia. No necesitas crear un objeto para usar métodos o atributos estáticos

```
public class Persona {  
    private String nombre;          // atributo de instancia  
    private static int contador;    // atributo de clase  
  
    public Persona(String nombre) {  
        this.nombre = nombre;  
        contador++;  
    }  
  
    public void saludar() { // método de instancia  
        System.out.println("Hola, soy " + nombre);  
    }  
  
    public static void mostrarContador() { // método de clase  
        System.out.println("Cantidad de personas: " + contador);  
    }  
  
    public static void main(String[] args) {  
        Persona p1 = new Persona("Ana");  
        Persona p2 = new Persona("Luis");  
        p1.saludar();  
        p2.saludar();  
        Persona.mostrarContador(); // Cantidad de personas: 2  
    }  
}
```

```
Persona.mostrarContador();
```

Variables Locales

¿Qué son?

Las variables locales se declaran **dentro de un método o bloque**. Solo existen mientras ese bloque se está ejecutando.

Regla importante

A diferencia de los atributos de clase, **no tienen valor por defecto**. Deben ser inicializadas antes de usarse o el compilador reportará un error.

```
public void mostrarEdad() {  
    int edad = 25; // variable local  
    System.out.println("Edad: " + edad);  
}  
// edad ya no existe aquí
```

Control de Flujo: Condicionales

Permiten que el programa tome decisiones y ejecute distintos bloques de código según condiciones booleanas.

if / else if / else

```
if (numero > 0) {  
    System.out.println("Positivo");  
} else if (numero < 0) {  
    System.out.println("Negativo");  
} else {  
    System.out.println("Cero");  
}
```

switch

```
switch (dia) {  
    case 1:  
        System.out.println("Lunes");  
        break;  
    case 2:  
        System.out.println("Martes");  
        break;  
    default:  
        System.out.println("Otro día");  
}
```

Ideal cuando se evalúa un único valor contra **múltiples casos** concretos.

Control de Flujo: Bucles

for

Número **determinado** de iteraciones. Se usa cuando se conoce cuántas veces repetir.

```
for (int i = 0; i < 5; i++)  
{  
    System.out.println(i);  
}
```

while

Repite mientras la condición sea **verdadera**. Puede no ejecutarse ninguna vez.

```
while (contador < 10) {  
    contador++;  
}
```

do-while

Ejecuta el bloque **al menos una vez** luego evalúa si debe continuar.

```
do {  
    leerEntrada();  
} while (!valido);
```

Conversión entre Tipos

Implícita (Widening)

De un tipo **más pequeño a uno más grande** Java lo hace automáticamente sin pérdida de información.

```
int entero = 100;  
double decimal = entero; // 100.0
```

Ruta: byte → short → int → long → float → double

Explícita (Casting)

De un tipo **más grande a uno más pequeño** Puede haber pérdida de información.
Requiere indicarlo explícitamente.

```
double decimal = 9.78;  
int entero = (int) decimal;  
// entero = 9 (se trunca)
```

⚠ La parte decimal se descarta sin redondeo.

Arreglos (Arrays)

¿Qué es un arreglo?

Estructura que almacena múltiples elementos de **un mismo tipo** en posiciones contiguas de memoria.
El acceso es por **índice comenzando en 0**.

Formas de declarar

```
// Con tamaño fijo
int[] numeros = new int[5];
numeros[0] = 10;

// Con valores iniciales
String[] nombres =
    {"Ana", "Luis", "Pedro"};
```

📌 El índice válido va de 0 hasta longitud - 1. Acceder fuera de rango lanza `ArrayIndexOutOfBoundsException`.

Arreglos Multidimensionales

Permiten representar **matrices** o **tablas** de datos. Se accede usando dos índices: `[fila][columna]`.

Declaración y acceso

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
  
System.out.println(  
    matriz[1][2]); // Imprime: 6
```

Visualización de la matriz

Col0	Col1	Col2
1	2	3
4	5	6

La celda resaltada corresponde a `matriz[1][2]`.

Fila 1, columna 2 → valor 6.

Tipos Wrapper (Clases Envoltorio)

Los tipos wrapper **encapsulan un primitivo dentro de un objeto**, permitiendo usarlos en colecciones y aprovechar métodos utilitarios.



`int` → `Integer`



`double` → `Double`



`char` → `Character`



`boolean` → `Boolean`

Autoboxing

```
Integer x = 5; // primitivo → objeto
```

Unboxing

```
int y = x; // objeto → primitivo
```

Primitivos vs. Wrappers

Aspecto	Primitivos	Wrappers
Almacenamiento	Stack (rápido)	Heap (más pesado)
Colecciones	✗ No directamente	✓ Sí (ArrayList, etc.)
Métodos	✗ Sin métodos	✓ <code>parseInt()</code> , <code>equals()</code> ...
Valor nulo	✗ No pueden ser <code>null</code>	✓ Pueden ser <code>null</code>

Usa primitivos cuando el rendimiento es crítico. Usa wrappers cuando necesites colecciones, métodos auxiliares o representar ausencia de valor.

Métodos Útiles de Wrappers



Conversión de Texto

Convierte `String` a número:

```
int n = Integer.parseInt("123");  
double d =  
Double.parseDouble("3.14");
```



Constantes de Límites

Valores máximo y mínimo de un tipo:

```
Integer.MAX_VALUE //  
2,147,483,647  
Integer.MIN_VALUE // -  
2,147,483,648
```



Clase Character

Métodos para analizar y transformar caracteres:

```
Character.isDigit('5'); // true  
Character.toUpperCase('a'); // 'A'
```

La Clase String

Características Esenciales

- Pertenecce a `java.lang` — **no requiere importación**
- Es **inmutable**: una vez creado, su valor no puede cambiar. Las "modificaciones" crean un nuevo objeto.

Métodos Clave

Método	Descripción
<code>length()</code>	Número de caracteres
<code>toUpperCase()</code>	Convierte a mayúsculas
<code>charAt(i)</code>	Carácter en posición i
<code>equals(otro)</code>	Compara contenido
<code>substring(a,b)</code>	Extrae subcadena

Constructores y Destructores

Constructor ✔

Se ejecuta **al crear** el objeto. Inicializa los atributos. No devuelve valor. Tiene el mismo nombre que la clase.

```
public Persona(String n) {  
    this.nombre = n;  
}
```

Destructor en Java ⚠

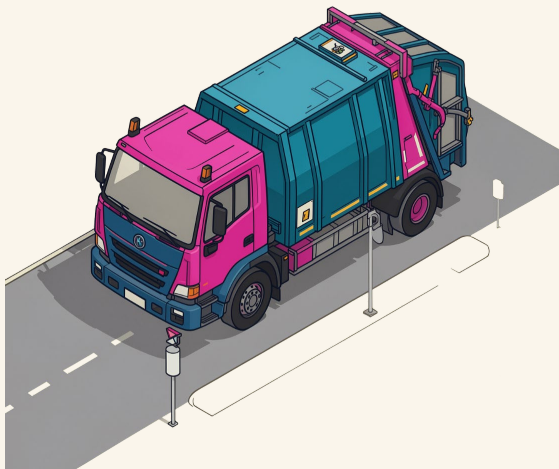
En Java **no existe un destructor real** como en C++. La liberación de memoria es gestionada automáticamente por el Garbage Collector.

Liberación de Recursos

Para archivos, sockets y otros recursos externos, Java usa `close()` o el bloque `try-with-resources`, que cierra automáticamente.

```
try (FileReader f =  
    new  
    FileReader("file.txt")) {  
    // uso del archivo  
} // se cierra solo
```

El Garbage Collector



¿Cómo funciona?

El Garbage Collector es un proceso automático de la JVM que **libera la memoria** ocupada por objetos que ya no tienen ninguna referencia activa.

→ Objetos en el Heap

Todo objeto creado con `new` vive en el **Heap** de memoria.

→ Sin referencias → "basura"

Cuando ninguna variable apunta al objeto, queda disponible para ser recolectado.

→ Recursos externos

⚠ El GC **no cierra** archivos, sockets ni conexiones. Usa siempre `close()` o `try-with-resources`.

Clases Abstractas

Una clase abstracta es una **plantilla base** que no puede instanciarse directamente. Define una estructura común para sus subclasses.

Características

- **Métodos abstractos:** sin implementación — las subclasses *deben* implementarlos
- **Métodos concretos:** con implementación — las subclasses los heredan directamente
- Se usa la palabra clave `abstract` en la clase y en cada método sin cuerpo

```
abstract class Vehiculo {  
  
    // Método abstracto  
    public abstract void  
    acelerar();  
  
    // Método concreto  
    public void encender() {  
        System.out.println(  
            "Motor encendido");  
    }  
}  
  
class Auto extends Vehiculo  
{  
    public void acelerar() {  
  
        System.out.println("Aceleran  
do");  
    }  
}
```

Interfaces

¿Qué es una interfaz?

Define un **contrato**: especifica *qué* debe hacer una clase, sin determinar *cómo* lo hace. Es la base de la herencia múltiple de comportamiento en Java.

Puntos clave

- Una clase puede implementar **múltiples interfaces**
- Se usa la palabra clave `implements`
- Todos los métodos son implícitamente `public abstract`

Ejemplo

```
interface Volador {  
    void despegar();  
    void aterrizar();  
    void volar();  
}  
  
class Avion extends Vehiculo  
    implements Volador {  
  
    public void despegar() { ... }  
    public void aterrizar() { ... }  
    public void volar() { ... }  
    public void acelerar() { ... }  
}
```

`Avion` hereda de `Vehiculo` e implementa `Volador`.

Clases Abstractas vs. Interfaces

Característica	Clase Abstracta	Interfaz
Métodos	Abstractos y concretos	Solo declaraciones
Variables	Atributos de instancia	Solo constantes
Constructores	✓ Sí	✗ No
Herencia	Una sola clase	Múltiples interfaces



Usa una **clase abstracta** cuando las subclases compartan código común. Usa una **interfaz** cuando solo necesites definir un contrato de comportamiento.

Composición y Agregación

Agregación

La "parte" **puede existir sin el "todo"**. El objeto relacionado se crea fuera y se pasa al constructor. Si el contenedor se destruye, la parte sigue viva.

```
// El conductor existe independiente
public Coche(Conductor c)
{
    this.conductor = c;
}
```

Composición

La "parte" **no puede existir sin el "todo"**. Se crea dentro del constructor. Si el contenedor se destruye, la parte también desaparece.

```
// El motor se crea dentro del Coche
public Coche(String marca) {
    this.motor = new Motor();
}
```

Próximos Pasos en Java



Practicar Fundamentos OOP

Resolver ejercicios con clases, herencia y polimorfismo para consolidar los conceptos de esta clase.



Explorar Colecciones

Aprender `ArrayList`, `HashMap` y `HashSet` para trabajar con grupos de datos de forma eficiente.



Manejo de Excepciones

Dominar `try-catch-finally` y `try-with-resources` para escribir programas robustos y seguros.

